

FPS Benchmark

- [Introduction](#)
- [Purpose](#)
- [Assumptions](#)
- [Metrics Collection Format](#)
 - [Benchmarking Conditions](#)
- [iOS Benchmarking Results](#)
 - [Impact on App Vitals](#)
 - [iPhone 13 Pro Max](#)
 - [iPhone 11](#)
- [Video](#)
- [Inference For iOS](#)
- [Android FPS](#)
 - [Metrics Collection Format / Methodology / Assumptions](#)
 - [ReactFps](#)
 - [Dependencies](#)
 - [Performance reports](#)
 - [Observations](#)
 - [Inference](#)

Introduction

This document aims to describe the steps taken to arrive at FPS numbers for main and JS Thread as a precursor to developing Anomaly Detection Framework on iOS and Android.

Purpose

This document would describe in detail the assumptions, results and inferences drawn on FPS numbers wrt to platform specific tools available on the platform. The deviations of the custom FPS numbers should be negligible from that of the Platform standard way of calculating FPS.

Assumptions

- JS FPS can only be calculated if JS Bridge is available as reference object.
- The POC is relevant to iOS / Android platform only.
- POC would be considered successful if the deviations are less than 0.5% wrt expected metrics.
- FPS numbers should also take into consideration the refresh rate of the devices.

Metrics Collection Format

FPS would be collected in relation to the device refresh rate and not capped at 60. FPS would be collected for Experiment and Control both. FPS would be collected using Xcode instruments in Control bucket and using custom code in Control bucket. Results from both the collections would be compared and discarded if deviation is greater than 0.5%.

Benchmarking Conditions

- Other apps should not be running on device.

- Device charge and thermal status should be same while collecting data in both experiment and control bucket.
- The data would be collected on two devices - one being a high end device and the other on lower end.
- The data would be collected for Home, PDP and PLP page.
- The average FPS numbers are rounded off to the nearest integer.
- Average FPS is calculated by taking an average of 3 iterations over each scenario.
- Only the FPS that lie within the scenario are considered and not the FPS when device is static as this would make it biased towards higher end.

iOS Benchmarking Results

Impact on App Vitals

	Standard App	App with FPS Measurement
CPU	3%-13%	3%-13%
Memory	76.8 MB- 82 MB	76.8 MB - 83 MB

iPhone 13 Pro Max

iPhone 13 Pro Max	FPS UI			FPS JS			Delta FPS UI			Delta FPS JS		
	Avg	Max	Min	Avg	Max	Min	Avg	Max	Min	Avg	Max	Min
Home-Control	73	78	71	72	76	70	-	-	-	-	-	-
Home-Test	73	78	71	72	76	69	0	0	0	0	0	-1
PDP-Control	70	78	43	68	76	31				-	-	-
PDP-Test	69	77	41	67	76	30	-1	-1	-2	-1	0	-1
PLP-Control	70	78	40	70	76	31				-	-	-
PLP-Test	70	78	41	69	77	31	0	0	1	-1	1	0

iPhone 11

iPhone 11	FPS UI			FPS JS			Delta FPS UI			Delta FPS JS		
	Avg	Max	Min	Avg	Max	Min	Avg	Max	Min	Avg	Max	Min
Home-Control	58	60	53	56	60	50	-	-	-	-	-	-
Home-Test	58	60	52	56	60	51	0	0	-1	0	0	1
PDP-Control	56	60	40	56	60	32				-	-	-
PDP-Test	56	60	42	56	60	31	0	0	2	0	0	-1
PLP-Control	58	60	50	57	60	47				-	-	-
PLP-Test	58	60	51	57	60	46	0	0	1	0	0	-1

Video



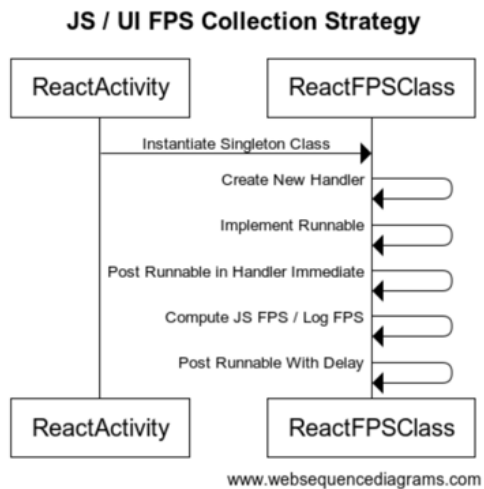
Inference For iOS

The results observed on both High end and medium end devices are within acceptable limits.

Android FPS

Metrics Collection Format / Methodology / Assumptions

1. FPS were collected based on Basic flows which the user did. Flows covered were Home Page, PDP, PLP
2. The FPS Collection is done on a separate runnable thread, which keeps posting / logging Data at a Regular interval
3. A balance between accuracy and Perf impact was taken to decide



ReactFps

```
1 package com.myntra.android.fps;
2
3
4 import com.facebook.react.bridge.ReactContext;
5 import com.facebook.react.modules.debug.FpsDebugFrameCallback;
6 import com.myntra.android.misc.L;
7
8 import android.os.Handler;
9
10 public class ReactFps {
11     private static ReactFps instance;
12     private Handler handler;
13     private FpsDebugFrameCallback mFrameCallback;
14
15     private final int DELAY = 50;
16
17     public static synchronized ReactFps getInstance() {
18         if (instance == null) {
19             instance = new ReactFps();
20         }
21
22         return instance;
23     }
24
25     public void init(ReactContext reactContext) {
26         mFrameCallback = new FpsDebugFrameCallback(reactContext);
27         handler = new Handler();
28     }
29
30     public void start(){
31         mFrameCallback.start();
32         handler.post(logFps);
33     }
```

```

34     }
35
36     public void stop() {
37         mFrameCallback.stop();
38         handler.removeCallbacks(logFps);
39     }
40
41
42     private final Runnable logFps = new Runnable() {
43         @Override
44         public void run() {
45             double jsFPS = mFrameCallback.getJSFPS();
46             double uiFPS = mFrameCallback.getFPS();
47
48             L.i("Logging FPS of App :: JS Fps" + jsFPS + " :: " + "UI FPS " + uiFPS);
49
50             handler.postDelayed(this, DELAY);
51         }
52     };
53 }
54
55
56

```

Dependencies

1. The implementation is dependant on React Native libraries, and is dependant on the **FpsDebugFrameCallback** class along with **reactContext**
2. Input Parameters can include the sampling / logging interval.

Performance reports

Device	Samsung M32					
	CPU Percentage	Memory Percentage	UI FPS	JS FPS	Sensitivity Variance	Sampling Interval
Home Page	27	320	70-90	70-90	10%	200
	28	320	85-90	67-80	5	100
	32	360	88-90	65-80	2%	50
	40	365	88-90	60-75	0%	10
	CPU Percentage	Memory Percentage	UI FPS	JS FPS	Sensitivity Variance	Sampling Interval
PLP / PDP Pages	54	500-700	70-90	60-80	10%	200
	57	500-700	85-90	67-80	5	100

	60	600-800	88-90	67-80	2%	50
	69	600-700	88-90	50-75	0%	10

Sensitivity Variance, is How fast the FPS changes occur wrt reacts FPS Monitor

Observations

1. Upon decreasing the sampling interval the the sensitivity is almost zero, i.e Even minute differences in in FPS can be captured if need be, incase we need a more fine grained FPS then this is useful
2. However there is a direct effect on the process CPU, the lesser the sampling interval the more the CPU consumption.

Based on the Above observations some tests were done to bench mark the above config, each page had 5 iterations and averages were put out.

Config: 100ms Sampling

Device: Samsung M32

Flows	Control CPU without FPS	CPU With FPS Enabled
Home Page	28	29
Shop Page	32	31
PLP	45	46
PDP Page	50	52

Inference

Above approach seems fine to implement JS and UI FPS with a sampling interval of 100ms.